SIMATS ENGINEERING

PYTHON PROGRAMMING ASSIGNMENT 4

Robust Data Processing with Files, Exceptions, Modules and Packages (CO4)

Student	Varshan J S
Register Number	1924260130A
Institution	SIMATS Engineering
Course Outcome	CO4 – File handling, exceptions, modules and packages
Python version	Python 3.13.5 in report-generation environment; student previously reported Python 3.14.2 on Windows
GitHub repository	CSA08-1924260130A-Assignment4 (add URL after repository creation)

Report scope

Six tasks covering file-processing diagnosis, biometric attendance aggregation, exception design, file forensics, four software organisations, and a modular college-fest registration package.

The project includes Python source files, sample input data, generated reports, audit logs, execution evidence and a README. Reference timings are labelled with the environment in which they were measured.

Assignment Overview

Task	Title	Marks	Evidence
1	Diagnose and repair poor file-processing script	6	Defects, robust parser, Decimal totals, rejected rows
2	Biometric attendance files	8	Monthly aggregation, two output files, trace
3	Exception handling in attendance processor	9	Failure table, custom exceptions, exception chaining
4	File and exception forensics	7	Defect reproduction, append repair, error log
5	Same utility organised four ways	7	Script, module, modules folder and package
6	College Fest Registration Desk	13	Package design, validation, duplicate handling, test cases

General processing strategy

Use explicit UTF-8 encoding and a context manager for each file. Stream large inputs line by line, validate a record before updating totals, catch expected exceptions near the operation that can raise them, and retain rejected rows with filename/line/reason. Use write mode for fresh reports and append mode for cumulative logs.

Reliable sequence: interpret the requirement → select file operations → parse and validate → aggregate → log failures → write outputs → test missing, damaged and large inputs.

Task 1 – File-Processing Design Review: Diagnose and Repair a Poor Script (6 Marks)

The hostel office script totals mess bills from lines containing register number, name and amount. The original code hard-codes a desktop path, reads the entire file, leaves the source open, uses a bare except/pass, assumes integer amounts, repeatedly concatenates report strings and fails to record rejected rows.

Defect	Operational consequence	Repair
Hard-coded absolute path	Fails on another PC or after file relocation	Take filename as a function parameter
File never closed	Descriptor leak; buffers may not flush	Use `with open(...)`
f.read() for all rows	Memory grows with file size	Iterate one line at a time
Bare except/pass	Hides malformed data and programming defects	Catch only expected parse/value errors
int(amount)	Cannot correctly handle paise/decimal amounts	Use Decimal for currency
Repeated string +	Repeated allocation and copying	Collect rows and write once
Blank row/no reject log	Index error or untraceable lost record	Skip blanks; log line, text and reason

Repaired design

`process_mess_file(file_name)` accepts the path as an argument, opens it with UTF-8 encoding and iterates using `enumerate()`. Each nonblank line is parsed as CSV and must contain exactly three fields. Register and name are required. The amount is converted to Decimal and must be finite and non-negative. Rejected lines are retained with line number, raw content and reason. `write_report()` writes the accepted rows once.

Reference execution and measured comparison

The sample produced 3 accepted records, 2 rejected records and a total of INR 3,681.25. The rejected rows were an invalid decimal amount and a row containing only two fields. A 50,000-line test compares line iteration with readlines():

Method	Elapsed time	Peak traced memory
Line iteration	0.013180 s	21,961 bytes
readlines()	0.032339 s	4,096,719 bytes

Both are O(n) in time. Iteration uses O(one line) auxiliary memory, while readlines() uses O(file size). In this run, readlines() used substantially more peak memory. The exact timings depend on the machine and runtime.

Task 2 – From Requirement to File-Processing Design: Biometric Attendance Files (8 Marks)

Input: daily UTF-8 text files containing `register_number,YYYY-MM-DD,P` or `register_number,YYYY-MM-DD,A`. Output: monthly attendance percentages, students below 75%, and damaged-line records containing filename and line number.

Data model and operations

A dictionary maps each register number to `[present_count, absent_count]`. List daily files with `Path.glob('*.txt')` and sort them. Open each file in `r` mode with UTF-8 and iterate line-by-line. Write the combined percentage/below-threshold report and a separate damaged-line CSV using `w` mode and context managers.

Processing pseudocode

```
LIST and sort the daily files
CREATE counts dictionary and errors list
FOR each file, OPEN and iterate lines with line numbers
SKIP blank lines
SPLIT line into register, date, status
IF field count is not 3 OR status not in {P,A}: append error and CONTINUE
INCREMENT present or absent count for register
FOR each register: compute present / (present + absent) × 100
WRITE the full percentage and below-75% report
WRITE damaged-line file
```

Trace example

For `192426003,2026-10-03,L`, the row has three fields but status L is invalid. The processor records the filename, line number 3, raw row and reason; no attendance count changes. It then continues to the following line. The next malformed one-field line is also recorded.

Reference execution

The sample files produced 192426001 = 75.00%, 192426002 = 50.00%, and 192426003 = 66.67%. The below-75% students are 192426002 and 192426003. Two damaged lines were logged. Reading each line once costs O(f × l), where f is file count and l is average lines per file; aggregation uses O(s) space for s students plus O(e) for e rejected rows. Re-reading every file for every student costs O(s × f × l).

Task 3 – Design the Error Handling: Exceptions in the Attendance Processor (9 Marks)

Failure	Typical exception	Handling
Missing folder/file	FileNotFoundError / NotADirectoryError	Log path; skip optional file or stop for essential source
Permission denied	PermissionError	Log file-level error; do not claim success
Invalid encoding	UnicodeDecodeError	Log decode failure and retain source for correction
Bad field count/status	InvalidRecord (custom)	Log line number and continue
Bad date or numeric conversion	ValueError / InvalidOperation	Translate at validation boundary using `raise ... from exc`
Programmer uses invalid index	IndexError	Treat as program defect; do not suppress silently

Custom exception and complete try statement

`InvalidRecord` represents invalid attendance content. `parse_line()` checks exactly three fields, required register/date values, ISO date format and P/A status. A ValueError from date parsing is translated with `raise InvalidRecord(...) from exc`, preserving the cause. `AttendanceFileError` wraps an OS error when a source cannot be opened.

The processor uses try for the risky operation, specific except clauses for expected failures, else for successful processing and finally for cleanup. A context manager is usually preferable because it guarantees closure. The misuse experiment shows that `except Exception: pass` hides a misspelled variable's NameError and allows the program to continue without any warning.

If malformed rows are common, explicit validation with if checks avoids frequent exception construction. A try/except parser is concise and centralises validation. Both remain O(n) over n rows, but constants differ: exception-based control flow is more costly for each invalid row. When invalid rows are rare, the additional cost is usually small.

Task 4 – File and Exception Forensics (7 Marks)

The original library program opens the master return log in write mode each day, writes book names without newline delimiters, manually closes files and reads a complete daily file. It only catches FileNotFoundError and prints a generic message.

Defect	Observed effect	Correction
Master log opened in `w`	Earlier days are truncated	Use append mode `a`
No newline after a book	Titles run together	Write each title plus `\n`
Manual close()	Exception may bypass close	Use context managers
Whole-file read	Memory scales with file size	Stream lines for large files
Generic missing message	Loses filename and cause	Append specific error details

Three-day evidence

The buggy log after three days contained only the final day's books, joined as `Book DBook E`. The repaired append-based log preserved all five book titles on separate lines. A deliberately missing file was recorded in the error log with its path and OS error.

Complexity and prevention

For b returned-book lines, reading and writing take O(b) time. Reading the whole file requires O(file size) working memory; streaming requires O(one line). File modes do not change asymptotic complexity. A context manager has the same normal-case time cost as open/close but reliably closes handles on all exits. Manual multi-file cleanup may leave multiple handles open if an exception interrupts control flow; with blocks close each file when its block exits.

Prevention rules: always use `with open(...)` with an explicit mode and encoding; use `a` for cumulative logs and include one newline per record; catch and log specific exceptions rather than hiding them.

Task 5 – The Same Utility Organised Four Ways (7 Marks)

The attendance utility reads records, calculates attendance percentages and identifies students absent for three consecutive days, then writes two reports. The same sample data is used in all four structures.

Structure	Organisation	Advantages	Adding the second report
One script	All functions in one file	Simple for small one-off work	Modify one file; high coupling
One module + main	attendance.py imported by main.py	Reusable functions; smaller main	Change module and possibly caller
Three modules	reader.py, stats.py, report.py	Separation and isolated tests	Change stats/report and orchestration
Package	attendance/__init__.py plus submodules	Public API and clean namespace	Change module; export API if needed

All four implementations returned matching results: 192426001 had 50.00% attendance; 192426002 had 25.00%; and 192426002 was absent for three consecutive dates.

Import forms

`import attendance.stats` is called as `attendance.stats.percentages(...)`; `from attendance import stats` is called as `stats.percentages(...)`; `from attendance.stats import percentages` is called directly as `percentages(...)`. Python caches imported modules in `sys.modules` after first load, so subsequent imports normally reuse the existing module. A package has a small one-time import and memory cost.

The package design is the strongest choice for a utility expected to grow: concerns are separated, modules can be tested independently and the public API can remain stable. A single script may be simpler for a tiny one-off task. Use specific module names to avoid shadowing standard-library modules.

Task 6 – Design Challenge: College Fest Registration Desk (13 Marks)

Stage 1 – Inputs, outputs and rules

Inputs are five department text files (CSE, ECE, EEE, MECH and CIVIL). Each row contains register number, name, department, e-mail and semicolon-separated events. Some rows may be duplicated, contain invalid e-mail, be damaged, or belong to a missing department file. Outputs are a clean master list, one participant file per event, a summary report and an error log. A bad row or absent department file must not stop other inputs.

Testable rules: exactly five fields; required identity fields nonempty; department matches source filename; e-mail passes string-method checks; at least one event; register number is the unique student key; duplicate register numbers merge event sets; all rejected or duplicate rows are audited.

Stage 2 – Processing flow

```
FOR each department file: attempt to open and stream lines
FOR each nonblank line: parse and validate fields
IF invalid: log filename, line, raw row and reason; CONTINUE
IF register number is already in master dictionary: merge its event set and audit duplicate
ELSE: create master record
AFTER all files: write master CSV, event-wise CSVs, summary and error log
```

Stage 3 – Critical design decisions

A dictionary keyed by register number detects duplicates with average O(1) lookup rather than scanning all previous registrations. Event choices are stored in a set to eliminate repeated event names. E-mail validation uses string methods: trim spaces, require one @, nonempty local/domain parts, no whitespace and a dot in the domain. This is an assignment-level validation, not a complete implementation of every e-mail standard. File-level errors are caught at the open/read boundary; content errors use the custom InvalidRegistration exception. Invalid rows are logged and processing continues.

Stage 4 – Package structure

Component	Responsibility
fest_package/reader.py	Read UTF-8 department files and yield line number/raw line
fest_package/validator.py	Registration model, InvalidRegistration and row/e-mail validation
fest_package/processor.py	Process departments, merge duplicates, produce event indexes and outputs
fest_package/__init__.py	Expose process_departments as the package API
main.py	Select paths/departments and invoke the package

Stage 5 – Implementation and demonstration

The five sample department files demonstrate clean registrations, one duplicate across departments, an invalid e-mail, a damaged row and a missing file test. The main run accepted 10 unique students and created five event lists: Coding (3), Dance (4), Music (4), Photography (3) and Robotics (3). The invalid e-mail, duplicate and malformed row were recorded. A separate test deliberately omits CIVIL: the missing file is logged and the other four files still process.

Alternative design and complexity

For n registration rows and d distinct register numbers, dictionary/set processing takes O(n) expected time and O(d + event memberships) space. Searching a list of earlier registrations for every incoming row can take $O(n^2)$ time. A set/dictionary of seen register numbers is the key optimisation. Event sets also deduplicate event choices.

Overall Evaluation and Submission Notes

Task	Time complexity	Additional memory
1	O(n) to parse n lines	O(one line) streaming plus results
2	O(total input lines)	O(students + rejected rows)
3	O(n) for n rows	O(invalid rows)
4	O(b) for b book lines	O(one line) streaming
5	O(n) processing in each design	Depends on retained input/results
6	O(n) expected via hash lookups	O(students + events + errors)

Execution validation

All six task implementations were executed successfully in the preparation environment. The four Task 5 organisations produced matching results. Task 6 produced the master/event/summary/error outputs and passed a missing-department test. Evidence logs and input files are included in the project. The reference environment was Python 3.13.5 on Linux x86_64; the student's VS Code terminal previously reported Python 3.14.2 on Windows. Re-run programs on the submission computer and use those timings for a viva.

Conclusion

Reliable file-processing programs expect imperfect data. Streaming, explicit encoding, context-managed file operations, Decimal currency values, specific exception handling, domain validation and structured error logs protect correctness and traceability. Separating reader, validator, processor and reporting functions into a package makes the college-fest registration solution easier to test, extend and maintain.

GitHub repository

Create a separate repository named CSA08-1924260130A-Assignment4, then add its complete URL to the cover page and README before final submission.